

AI SOFTWARE QA PORTFOLIO

AI 기반 RAG 챗봇
자동 품질 평가 및 답변 개선 시스템
Evaluation → Correction → Reporting Feedback Loop

RAG · LLM-as-a-Judge · Correction Agent · QA 자동화

프로젝트 기간	2026.06
분야	AI 품질 평가(QA) / 답변 자동 개선
핵심 역할	평가·개선 파이프라인 설계 및 Agent 구현
주요 기술	Python · LangChain · GPT-4.1-mini · ChromaDB
핵심 산출물	자동 채점 + 모범답안 생성 + Markdown 리포트

목 차

1. 프로젝트 개요

구분	내용
프로젝트명	AI 기반 RAG 챗봇 자동 품질 평가 및 답변 개선 시스템
프로젝트 기간	2026.06
한 줄 요약	RAG 챗봇 답변을 자동 평가하고, 실패 답변을 모범답안으로 자동 수정하는 QA 피드백 루프 시스템
핵심 목표	평가 → 분석 → 개선 → 보고서 생성으로 이어지는 AI 품질관리(QA) 자동화
평가 방식	Two-Stage Evaluation (루브릭 평가 + 감점 평가 분리)
핵심 산출물	평가 리포트(JSON) · 수정된 모범답안(JSON) · Markdown 종합 보고서

■ 프로젝트 배경

기존 AI 챗봇 평가는 사람이 직접 답변을 확인하고 점수를 부여하는 방식이 대부분이었다. 이 방식은 평가자의 주관에 개입될 수 있고, 동일한 기준을 유지하기 어려우며, 테스트 건수가 늘어날수록 많은 시간이 소요되는 문제가 있다.

이를 해결하기 위해 GPT 기반 Judge Agent를 활용한 자동 평가 시스템을 구축했다. 여기서 한 단계 더 나아가, 평가 결과를 다시 답변 개선에 활용할 수 있도록 Correction Agent를 추가하여, 낮은 점수를 받은 실패 사례를 자동으로 수정된 모범답안으로 변환하도록 설계했다. 즉 본 프로젝트는 ‘채점’에서 멈추지 않고 ‘개선’까지 이어지는 피드백 루프를 목표로 한다.

2. 시스템 아키텍처

사용자 질문이 입력되면 RAG 챗봇이 ChromaDB 에서 유사 문서를 검색하고, GPT-4.1-mini 가 이를 근거로 답변을 생성한다. Judge Agent 가 답변을 Two-Stage 방식으로 평가하여 JSON 리포트를 만들고, 이 리포트를 입력받은 Correction Agent 가 수정된 모범답안을 생성한다. 최종적으로 모든 결과가 Markdown 보고서로 정리된다.

- ① **사용자 질문** 질문 입력
- ▼
- ② **RAG 챗봇** 검색·생성 오케스트레이션
- ▼
- ③ **Vector DB (ChromaDB)** 유사 문서 검색
- ▼
- ④ **GPT-4.1-mini** 근거 기반 답변 생성
- ▼
- ⑤ **Judge Agent** Two-Stage 품질 평가
- ▼
- ⑥ **평가 리포트 (JSON)** 점수·감점 사유 구조화
- ▼
- ⑦ **Correction Agent** 실패 답변 자동 수정
- ▼
- ⑧ **수정된 모범답안 (JSON)** 개선 답변·근거 생성
- ▼
- ⑨ **Markdown 보고서** 종합 결과 자동 문서화

아키텍처 포인트

⑥~⑧ 구간이 본 시스템의 핵심 차별점이다. 평가에서 멈추지 않고, 평가 결과(JSON)를 그대로 Correction Agent 의 입력으로 재사용하여 개선까지 자동으로 이어지도록 설계했다.

3. 담당 역할

- RAG 챗봇 품질 평가 프로세스 설계
- Two-Stage Evaluation(TSE) 평가 방식 설계
- GPT 기반 Judge Agent 구현
- GPT 기반 Correction Agent 구현
- Markdown 자동 보고서 생성 기능 구현
- 테스트 케이스 작성 및 자동 실행 환경 구축
- 품질 개선 피드백 루프(Evaluation → Correction → Reporting) 설계

4. 기술 스택

구분	기술	역할
Language	Python	전체 파이프라인 구현
LLM	OpenAI GPT-4.1-mini	답변 생성·평가·수정
Framework	LangChain	RAG 파이프라인 구성
Vector Database	ChromaDB	유사 문서 저장·검색
Embedding	text-embedding-3-small	문서·질문 임베딩
Prompt Engineering	Few-shot Prompt, TSE Prompt	평가·수정 프롬프트 설계
환경 관리	python-dotenv	API 키 등 환경 설정 관리
문서 처리	PyPDFLoader, TextLoader	PDF·텍스트 문서 로드
결과 출력	JSON, Markdown	구조화 데이터 및 리포트 산출

5. 프로젝트 구조

평가(Evaluator)와 개선(Correction) 로직을 별도 모듈로 분리하여 책임을 명확히 나눈 구조다.

```
rag_chatbot/
├─ evaluation_prompt.py  # 평가 프롬프트(TSE) 정의
├─ evaluator_agent.py    # Judge Agent 실행 로직
├─ correction_prompt.py  # 수정 프롬프트 정의
├─ correction_agent.py   # Correction Agent 실행 로직
├─ rag_service.py        # 검색·답변 생성 서비스
├─ report.py             # Markdown 보고서 생성
├─ run_tests.py          # 테스트 자동 실행
├─ test_cases.json       # 테스트 케이스 정의
├─ chroma_db/            # 벡터 DB 저장소
└─ reports/              # 생성된 보고서 저장
```

6. 주요 기능

1

RAG 검색

사용자 질문과 가장 유사한 문서를 ChromaDB 에서 검색하여 GPT 에게 전달한다.

2

AI 답변 생성

검색된 문서를 기반으로 GPT 가 답변을 생성한다.

Hallucination 을 최소화하기 위해 검색된 문서를 우선적으로 활용하도록 설계했다.

3

Judge Agent

생성된 답변을 이해도 · 정확성 · 관련성 · 표현성 네 항목으로 자동 평가한다.

4

자동 점수 계산

루브릭 점수와 감점 점수를 반영하여 최종 점수를 자동으로 계산한다.

5

Correction Agent

평가 리포트를 기반으로 기존 답변과 감점 사유를 분석하고, 수정된 모범답안과 개선 이유를 자동 생성한다.

6

Markdown 자동 보고서

테스트 결과 · 루브릭 평가 · 감점 평가 · 최종 점수 · 종합 의견을 포함한 보고서를 자동 생성한다.

7. 평가 방식 — Two-Stage Evaluation

1 단계 · 루브릭 평가

항목	평가 내용
이해도	질문의 의도를 정확히 파악했는가
정확성	검색된 문서 근거에 기반한 답변인가
관련성	질문과 답변이 적합하게 연결되는가
표현성	가독성과 표현이 자연스러운가

2 단계 · 감점 평가

감점 규칙	감점
출처 미기재	-1
정의만 제공	-1
근거 문서 미사용	-1
문서 존재 시 부적절한 응답	-2

Correction Agent 출력 형식

평가 리포트를 입력받아 아래와 같은 구조화된 JSON 으로 수정 결과를 반환한다.

```
{
  "corrected_answer": "...",
  "reason_for_correction": "...",
  "applied_improvements": []
}
```

8. 테스트 결과

항목	결과
총 테스트 건수	5 건
평균 점수	4.20 / 5.00
평가 항목	이해도 · 정확성 · 관련성 · 표현성
자동 생성 보고서	rag_evaluation_20260625_150331.md

결과 해석

5 건의 테스트에서 평균 4.20 점을 기록했으며, 모든 평가와 보고서 생성이 자동으로 수행되었다. 특히 낮은 점수를 받은 답변은 폐기되지 않고 Correction Agent 를 거쳐 수정된 모범답안으로 전환되어, 평가 결과가 실질적인 개선으로 이어지는 구조를 확인했다.

9. 문제 해결 사례

문제 1

- **기존 방식** 점수만 제공되어 왜 감점되었는지 알 수 없음
- **해결** 루브릭 평가와 감점 평가를 분리하여 감점 원인을 명확하게 제공함

문제 2

- **기존 방식** 낮은 점수를 받은 답변이 그대로 폐기됨
- **해결** Correction Agent 를 구현하여 실패 사례를 수정된 모범답안으로 자동 변환함

문제 3

- **기존 방식** QA 보고서를 사람이 직접 작성해 시간이 소요됨
- **해결** Markdown 보고서를 자동 생성하도록 구현함

10. 프로젝트 성과

- ✓ GPT 기반 Judge Agent 구현
- ✓ Two-Stage Evaluation(TSE) 적용
- ✓ GPT 기반 Correction Agent 구현
- ✓ AI 답변 자동 개선 시스템 구축
- ✓ Markdown 자동 보고서 생성
- ✓ RAG 품질 평가 자동화
- ✓ QA 업무 자동화 기반 마련

■ 프로젝트를 통해 배운 점

이번 프로젝트를 통해 AI 품질관리는 단순히 점수를 부여하는 것이 아니라, 평가 결과를 다시 개선에 활용하는 과정이 중요하다는 점을 경험했다. Judge Agent 로 객관적인 품질 평가를 수행하고, Correction Agent 로 실패한 답변을 모범답안으로 변환하는 피드백 루프를 구축했다.

또한 Prompt Engineering, RAG, GPT API, ChromaDB 를 함께 활용하며, 실제 AI 서비스 품질관리 업무와 유사한 자동화 시스템을 직접 설계하고 구현해볼 수 있었다.

11. 향후 개선 계획

- 다중 LLM Judge(Ensemble Evaluation) 적용으로 단일 모델 편향 완화
- BLEU · ROUGE · BERTScore 기반 정량 평가 추가
- Streamlit 대시보드 구축으로 결과 가시성 향상
- 평가 결과 기반 자동 Prompt 개선
- Fine-tuning 학습 데이터 자동 생성
- AI QA 운영 대시보드 연동

12. 핵심 역량

AI Software QA	Prompt Engineering	RAG 시스템 평가
GPT API 활용	LangChain	ChromaDB
Python 개발	AI Agent 설계	테스트 자동화
품질 평가 및 개선 자동화		

프로젝트 요약

본 프로젝트는 RAG 챗봇 품질 평가를 자동화하는 데서 한 걸음 더 나아가, 평가 결과를 다시 답변 개선에 활용하는 Evaluation → Correction → Reporting 피드백 루프를 구현한 사례다. 채점에 머무르지 않고 실질적인 품질 개선까지 이어지는 자동화 구조를 통해, AI 서비스 품질관리(QA) 업무에 대한 실무적 이해를 보여준다.